

Principles of Programming Languages:

Problem Sheet 3 - Semantic variations

Wira Azmoon Ahmad

17 Nov 2023

Question 1

```
1  -- Q1
2  --  $xm = Ok\ x$ 
3  ( $xm\ \$>\ f$ )  $\$>\ g$ 
4  = (( $Ok\ x$ )  $\$>\ f$ )  $\$>\ g$       -- Definition of  $M\ x$ 
5  =  $f\ x\ \$>\ g$                 -- Definition of  $\$>$ 
6   $xm\ \$>\ (\backslash x \rightarrow f\ x\ \$>\ g)$ 
7  = ( $Ok\ x$ )  $\$>\ (\backslash x \rightarrow f\ x\ \$>\ g)$  -- Definition of  $M\ x$ 
8  =  $(\backslash x \rightarrow f\ x\ \$>\ g)(x)$       -- Definition of  $M\ x$ 
9  =  $f\ x\ \$>\ g$ 
10 --  $xm = Fail$ 
11 ( $xm\ \$>\ f$ )  $\$>\ g$ 
12 = ( $Fail\ \$>\ f$ )  $\$>\ g$       -- Definition of  $M\ x$ 
13 =  $Fail\ \$>\ g$               -- Definition of  $\$>$ 
14 =  $Fail$                     -- Definition of  $\$>$ 
15  $xm\ \$>\ (\backslash x \rightarrow f\ x\ \$>\ g)$ 
16 =  $Fail\ \$>\ (\backslash x \rightarrow f\ x\ \$>\ g)$  -- Definition of  $M\ x$ 
17 =  $Fail$                     -- Definition of  $\$>$ 
18
19 ( $result\ x$ )  $\$>\ f$ 
20 = ( $Ok\ x$ )  $\$>\ f$       -- Definition of  $result$ 
21 =  $f\ x$                 -- Definition of  $\$>$ 
22
23  $xm\ \$>\ result$ 
24 = ( $Ok\ x$ )  $\$>\ result$  -- Definition of  $M\ x$ 
25 =  $result\ x$           -- Definition of  $\$>$ 
26 =  $Ok\ x$               -- Definition of  $result$ 
27 =  $xm$                   -- Definition of  $M\ x$ 
28
29  $orelse\ failure\ ym = ym$  -- Definition of  $orelse$ 
30  $orelse\ ym\ failure = ym$  --  $ym$ , if  $ym = (Ok\ x)$  then  $ym$  else if  $ym = failure$  then  $ym$ 
```

Question 2

```
1  -- Q2a
2  result  :: a -> M a
3  result  x = Ok x
4
5  ($>) :: M a -> (a -> M b) -> M b
6  ($>) (Ok x) f = f x
7  ($>) (Fail v) _ = Fail v
```

```
1  -- Q2b
2  failure  :: Value -> M a
3  failure  v = Fail v
4
5  orElse  :: M a -> (Value -> M a) -> M a
6  orElse  (Ok x) _ = Ok x
7  orElse  (Fail v) f = f v
```

```
1  -- Q2c
2  eval (Failure v) env = failure (eval v env)
3  eval (OrElse e1 e2) env = orElse (eval e1 env) (\v -> eval e2 env $> (\fv -> apply fv [v]) )
```

(d) The expression gives 2. We can force the `eval e2 env` before `orElse`.

```
1  -- Q2d
2  eval (OrElse e1 e2) env = eval e2 env $> (\fv -> orElse (eval e1 env) (\v -> apply fv [v]) )
```

Question 3

```
1  ----- Q3
2  data Maybe a = Just a | Nothing
3
4  type M a = Mem -> Maybe (a, Mem)
5
6  -- Q3a
7  result  :: a -> M a
8  result  x mem = Just (x, mem)
9
10 ($>) :: M a -> (a -> M b) -> M b
11 ($>) xm f mem = let xa = xm mem in
12   case xa of
13     Just (x, mem') -> f x mem'
14     Nothing -> Nothing
15
```

```

16 -- Q3b
17 new :: M Location
18 new mem = let (a, mem') = fresh mem in result a mem'
19
20 get :: Location -> M Value
21 get a mem = result (contents mem a) mem
22
23 put :: Location -> Value -> M ()
24 put a v mem = result () (update mem a v)
25
26 failure :: M a
27 failure mem = Nothing
28
29 orElse :: M a -> M a -> M a
30 orElse xm ym mem = let xa = xm mem in
31   case xa of
32     Just (x, mem') -> Just (x, mem')
33     Nothing -> ym mem

```

Question 4

```

1 -----Q4
2 -- Q4a
3 result :: a -> M a
4 result x mem = (Just x, mem)
5
6 ($>) :: M a -> (a -> M b) -> M b
7 ($>) xm f mem = let (xa, mem') = xm mem in
8   case xa of
9     Just x -> f x mem'
10    Nothing -> (Nothing, mem')
11
12 -- Q4b
13 new :: M Location
14 new mem = let (a, mem') = fresh mem in result a mem'
15
16 get :: Location -> M Value
17 get a mem = result (contents mem a) mem
18
19 put :: Location -> Value -> M ()
20 put a v mem = result () (update mem a v)
21
22 failure :: M a
23 failure mem = (Nothing, mem)
24
25 orElse :: M a -> M a -> M a
26 orElse xm ym mem = let (xa, mem') = xm mem in
27   case xa of
28     Just x -> result x mem'
29     Nothing -> ym mem'

```

Question 5

(a) $M_1 \alpha$ maps the memory to a type *Maybe*, applied on a tuple of (α, Mem) while $M_2 \alpha$ maps the memory to a tuple, within which there's a *Maybe* and *Mem* type.

(b) This means that the first type keeps memory only if the *Maybe* type is *Just*, not if it is *Nothing*, while the second type always keeps memory whether or not the *Maybe* type is *Just* or *Nothing*. The first type is more efficient since it does not worry about memory as much.

(c) The example below has 1 for the first interpreter, and 2 for the second.

```
1 val x = new();;
2 put(x, 1);;
3 (x:=2; failure ) or else (get(x));;
```

Question 6

```
1 -- Q6
2 eval (Address x) env mem = eval x env mem
3 eval (Contents x) env mem = eval (Apply (Variable "!" ) [x]) env mem
4 etranslate :: Expr -> Expr
5 etranslate (Assign (Contents e1) e2) = Assign (etranlate e1) ( etranslate e2)
6 etranslate (Assign e1 e2) = Assign e1 ( etranslate e2)
7 etranslate (Variable x) = let v = maybe_find init_env x in
8   case v of
9     Just y -> Variable x
10    Nothing -> Apply (Variable "!" ) [Variable x]
11 etranslate (Apply (Variable f) es) = let v = maybe_find init_env f in
12   case v of
13     Just x -> Apply (Variable f) (map etranslate es)
14    Nothing -> Apply (etranlate (Variable f)) (map (ref . etranslate ) es)
15 etranslate (Lambda xs body) = Lambda xs (etranlate body)
16 etranslate (Sequence e1 e2) = Sequence (etranlate e1) ( etranslate e2)
17 etranslate (Let d e1) = Let (dtranlate d) ( etranslate e1)
18 etranslate e = e
19
20 dtranlate :: Defn -> Defn
21 dtranlate (Val x e) = Val x (ref ( etranslate e))
22 dtranlate d = d
23
24 ref :: Expr -> Expr
25 ref e = Apply (Variable "ref") [e]
```

Question 7

```

1  -- Q7a
2  val swap(a,b) = let val t = !a in a := !b; b := t;;
3
4  val x = new();;
5  x := 3;;
6  val y = new();;
7  y := 4;;
8  swap(x,y);;
9  !x;;
10 --> 4
11 !y;;
12 --> 3

1  -- Q7b
2  etranslate (Address x) = x
3  etranslate (Contents x) = Apply (Variable "!" ) [ etranslate x]

1  -- Q7c
2  val swap(a,b) = let val t = *a in *a := *b; *b:= t; t;;
3
4  val x = 3;;
5  val y = 4;;
6  x:y: nil ;;
7  --> [3, 4]
8  swap(&x, &y);;
9  --> 3
10 x:y: nil ;;
11 --> [4, 3]

1  -- Q7d
2  *(&x)
3  = Contents (Address (Variable x))
4  = Apply (Variable "!" ) [Variable x]
5  = Variable x

```

(e) No, since x may not be an address, so dereferencing it does not make sense.

```

1  -- Q7e
2  val x = 3;;
3  *(&x);;
4  --> 3
5  &(*x);;
6  Error: FunC.hs:149:20–58: Non-exhaustive patterns in lambda

```

Question 8

```

1  -- Q8
2  fibk :: Int -> (Int -> Ans) -> Ans
3  fibk n k = if n <= 1 then k n else fibk (n-1) (\r1 -> fibk (n-2) (\r2 -> k (r1 + r2)))

```

Question 9

```

1  -- Q9
2  (xm $> f) $> g
3  = (\k -> xm (\x -> f x k)) $> g -- Definition of $>
4  = (\k' -> (\k -> xm (\x -> f x k)) (\x' -> g x' k')) -- Definition of $>
5  = (\k' -> xm (\x -> f x (\x' -> g x' k))) -- Simplify lambda k
6  xm $> (\x -> f x $> g)
7  = \k -> xm (\x' -> (\x -> f x $> g) x' k) -- Definition of $>
8  = \k -> xm (\x' -> (f x' $> g) k) -- Simplify lambda x
9  = \k -> xm (\x' -> (\k' -> f x' (\x -> g x k')) k) -- Definition of $>
10 = \k -> xm (\x' -> (f x' (\x -> g x k))) -- Simplify lambda k'
11 = (xm $> f) $> g -- Set k = k', x' = x, x = x'
12
13 (result x) $> f
14 = \k -> (result x) (\x' -> f x' k) -- Definition of $>
15 = \k -> (\k' -> k' x) (\x' -> f x' k) -- Definition of result
16 = \k -> ((\x' -> f x' k) x) -- Simplify lambda k'
17 = \k -> f x k -- Simplify lambda x'
18 = f x -- Definition of f x
19
20 xm $> result
21 = (\k -> k x) $> result -- Definition of M x
22 = \k' -> (\k -> k x) (\x -> result x k') -- Definition of $>
23 = \k' -> ((\x -> result x k') x) -- Simplify lambda k
24 = \k' -> result x k' -- Simplify lambda x
25 = \k' -> k' x -- Definition of result
26 = xm -- Definition of M x

```